

STANDARD OPERATING PROCEDURE

How to design system architecture

REVISION HISTORY

Revision No. / Version No.	DATE	Prepared by:	Approved by:	Details of Change
1.0	20-June-2025	Vishal Vernekar (SD)	Mayur Pednekar (Manager IT)	

1. OBJECTIVES

Designing a system architecture and its accompanying design to ensure that the system is scalable, maintainable, and meets the requirements of its stakeholders.

2. PARTICIPANTS

1. Project Manager(PM): The PM assigned towards the department.
2. Business Analyst(BA): The BA lead
3. Solution Architect: To get the design done with TL
4. CTO: To review the system architecture.

3. PROCEDURE

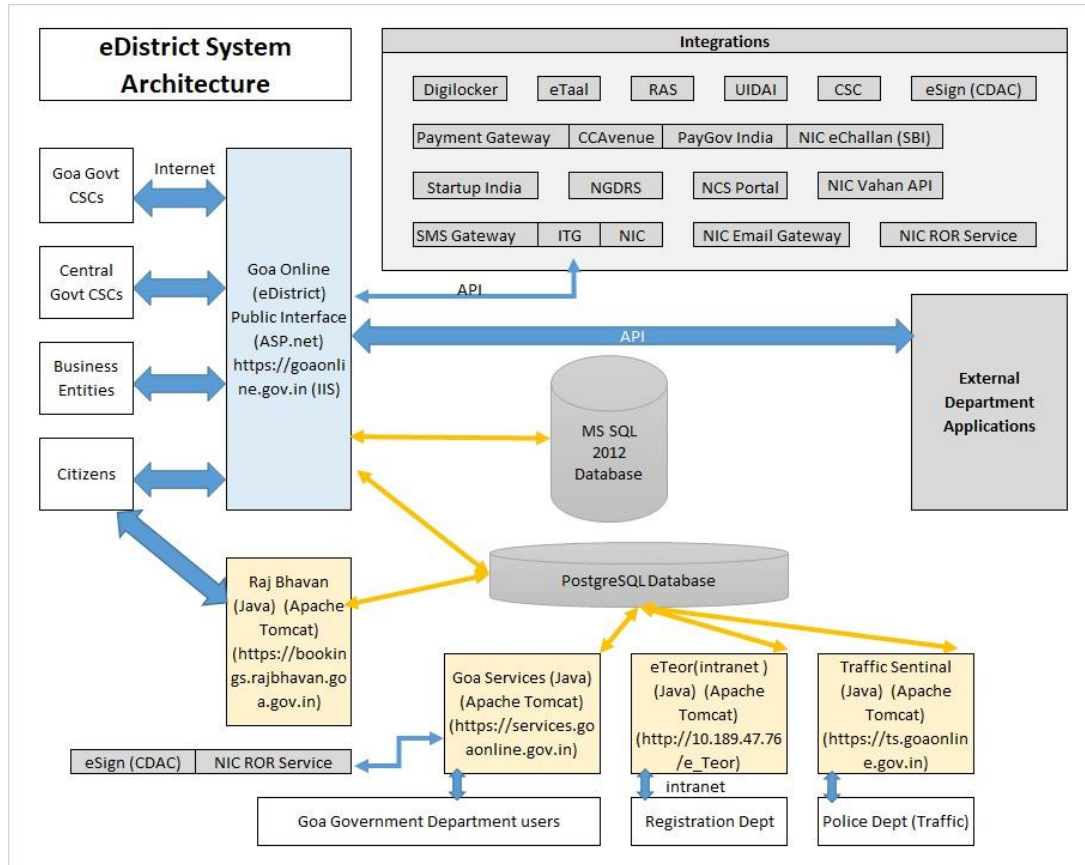
1. Define clear boundary of the project scope.
2. List out the stake holders and expected integration outside of the boundary.
3. Inside the boundary, define the following components
 - a) Hardware requirement with OS, system software with version.
 - b) API services if any, software being used with version.
 - c) DB services if any, software being used with version.
 - d) Show connection between various components using arrows/lines.
 - e) Define various component to be used in Internal integration
 - f) Include load balancers to isolate public network and high availability.
 - g) Plan for horizontal scaling by increasing the cluster of DBs and Application/API services.
 - h) If using ReactJS consider using caching services like Redis, Memcached.
 - i) If Planning for extensive document usage, consider having object storage like AWS S3 bucket.
 - j) Plan for firewall to prevent DDos.
4. Once the system architecture diagram is ready, take approval from CTO.

STANDARD OPERATING PROCEDURE

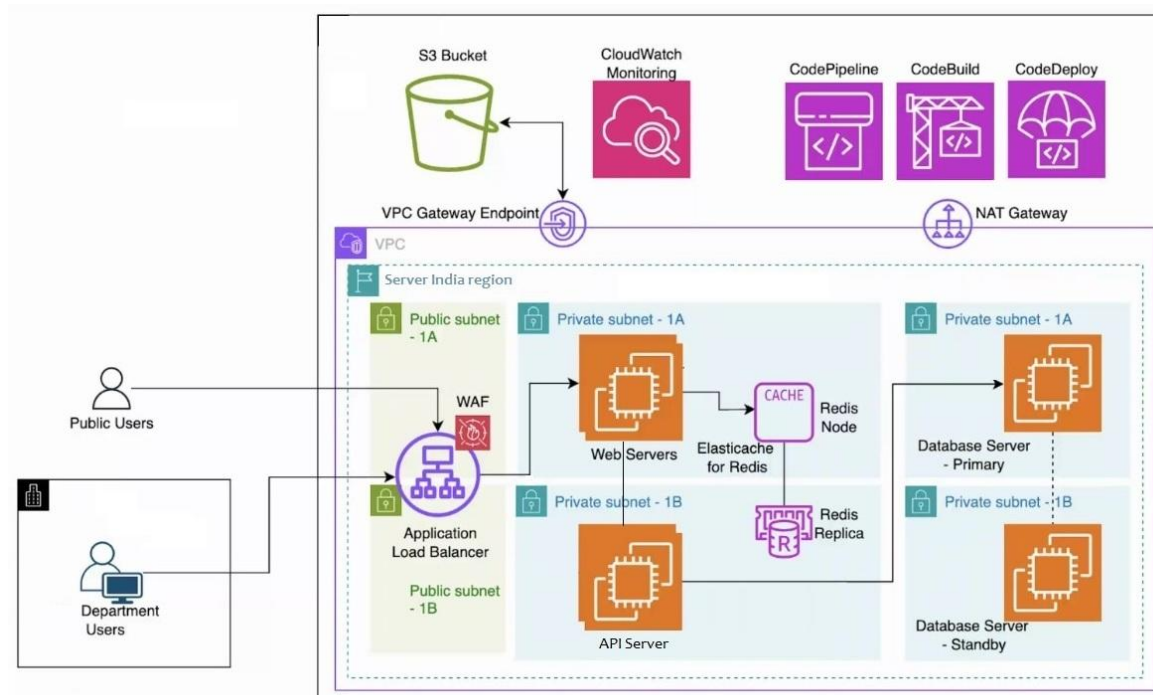
How to design system architecture

Sample architecture for reference:

1. eDistrict System Architecture:



2. eDistrict AWS Architecture:



4. REFERENCE SECTION: COMPONENTS DEFINATION AND DESCRIPTION

1. Users & Clients:
 - a) Represent different types of users (e.g., Admin, Regular Users).
 - b) Include client devices (e.g., Web Browser, Mobile App).
2. Load Balancer:
 - a) A component that distributes incoming traffic across multiple servers.
3. Frontend:
 - a) Depict the frontend as a separate component, representing the user interface.
 - b) Include connections to the web browser and mobile app.
4. Backend:
 - a) Application Servers: Illustrate one or more application servers that handle the business logic.
 - b) Microservices (if applicable): Show different microservices, each handling a specific aspect of the business logic.
 - c) APIs: Draw the APIs used by the frontend to communicate with the backend.
5. Database:
 - a) Represent one or more databases that store persistent data.
 - b) Include connections between the backend and the database.
6. Cache:
 - a) Illustrate a caching layer (e.g., Redis, Memcached) used to store frequently accessed data for quick retrieval.
7. External Services:
 - a) Show any external services or third-party APIs the system interacts with (e.g., payment gateways, authentication services).
8. Monitoring & Logging:
 - a) Depict tools or services used for monitoring and logging (e.g., Prometheus, ELK Stack).
9. CDN (Content Delivery Network):
 - a) Show a CDN that delivers static content like images, CSS, and JavaScript files to users.

10. Security Components:

- a) Represent security components such as firewalls, identity management systems, and encryption.

11. Development Environment:

- a) Include CI/CD pipelines, development, and staging environments.

5. TOOLS TO CREATE THE DIAGRAM

1. Lucidchart - User-friendly and has templates for system architecture diagrams.
2. Microsoft Visio - A powerful tool for creating detailed diagrams.
3. Draw.io (diagrams.net) - Free and open-source, integrated with Google Drive.
4. Miro - An online collaborative whiteboard platform with diagramming capabilities.